

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

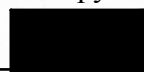
«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

Реализация системы защиты данных на уровне файловой системы и веб-интерфейса

Выполнили:

Жук Анастасия Валерьевна,
студентка группы N3348



(подпись)

Проверил:

Ищенко Алексей Петрович

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	4
Задание 1(а). Локальная программа.....	4
Техническое задание.....	4
Описание продукта, его функционал.....	5
Инструкция по применению.....	6
Код на языке программирования Python.....	7
Примеры работы программы.....	19
Задание 1(б). Веб-скрипт.....	27
Техническое задание.....	27
Описание продукта, его функционал.....	28
Инструкция по применению.....	29
HTML код с внедренным JavaScript кодом.....	30
Примеры работы программы.....	39
ЗАКЛЮЧЕНИЕ.....	46

ВВЕДЕНИЕ

Цель работы – исследовать и реализовать практические методы защиты информации на двух уровнях – на уровне файловой системы операционной системы и на уровне веб-интерфейса – с целью ограничения несанкционированного доступа, копирования и модификации данных.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- освоить на практике принципы контроля доступа к данным на разных уровнях (операционной системы и веба);
- изучить соответствующие технологии для реализации защиты;
- реализовать два независимых модуля защиты, демонстрирующих различные подходы к информационной безопасности.

ХОД РАБОТЫ

Задание 1(а). Локальная программа

Техническое задание

Разработать программу, запрещающую в текущем (том, котором она находится) каталоге создание, копирование, удаление или переименование файлов с заданными именами (можно использовать маски файлов). Список имен или их шаблонов хранить в файле `template.tbl` как текст. Должна быть обеспечена защита этого файла от удаления, несанкционированного просмотра и модификации. При установке программы можно предусмотреть ее отключение с использованием пароля, хранящегося в первой строке файла `template.tbl` в хешированном виде. Программа должна включать и выключать режим защиты.

Описание продукта, его функционал

Продукт `file_guard.py` представляет из себя программу, написанную на языке программирования Python для операционной системы Windows, которая устанавливается в папку, в которой необходимо обеспечить защиту файлов.

Защита применяется только к тем файлам, которые соответствуют маскам, записанным в файле `template.tbl`. Она блокирует открытие этих файлов, переименование, копирование и удаление. Также блокирует создание файлов, соответствующих маске.

После запуска создаются два файла: `file_guard.log` и `guard.lock`. В `file_guard.log` хранятся логи работы программы, а `guard.lock` нужен для того, чтобы закрыть скрывающуюся консоль после запуска. После снятия защиты `guard.lock` удаляется из папки, а `file_guard.log` – нет.

Инструкция по применению

Необходимо переместить программу file_guard.py в папку, в которой необходимо реализовать защиту файлов. Также в папке должен лежать файл с паролем для запуска в хешированном виде (в данной программе используется SHA-256) и набор масок файлов.

Запуск программы осуществляется через консоль с правами администратора. Через команду cd нужно перейти в защищаемую папку. В ней можно запустить программу следующей командой:

```
python file_guard.py
```

Появится информация о том, что может делать эта программа.

Запуск защиты

```
python file_guard.py start
```

Появится сообщение о том, что нужно ввести пароль для активации. Число попыток ввести пароль ограничено (3 попытки). После ввода правильного пароля, консоль пропадет. Программа будет работать в фоновом режиме.

Статус защиты

```
python file_guard.py status
```

Необходимо открыть новую консоль от имени администратора, перейти в защищаемую папку и запустить команду, написанную выше. В консоли появится информация о статусе защиты: включена или выключена, в какой папке действует и PID процесса защиты.

Снятие защиты

```
python file_guard.py stop
```

Необходимо открыть новую консоль от имени администратора, перейти в защищаемую папку и запустить команду, написанную выше. Появится сообщение о том, что нужно ввести пароль для активации. Число попыток ввести пароль ограничено (3 попытки). После ввода правильного пароля, защита перестанет действовать.

Код на языке программирования Python

```
import os
import sys
import hashlib
import fnmatch
from pathlib import Path
import logging
import ctypes
import win32security
import ntsecuritycon
import win32con
import time
import psutil
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import win32gui
import win32con as wcon
import subprocess

# Обработчик событий файловой системы для мониторинга изменений
class FileGuardEventHandler(FileSystemEventHandler):
    def __init__(self, file_guard):
        # Ссылка на основной объект защиты для вызова методов
        self.file_guard = file_guard

    def on_created(self, event):
        # Обработка создания нового файла
        if not event.is_directory and self.file_guard.matches_pattern(event.src_path):
            self.file_guard.delete_protected_file(event.src_path, "создание")

    def on_deleted(self, event):
        # Обработка удаления файла
        if not event.is_directory and self.file_guard.matches_pattern(event.src_path):
            self.file_guard.block_file_operation(event.src_path, "удаление")

    def on_moved(self, event):
        # Обработка перемещения/переименования файла
        if not event.is_directory:
            # Если целевой файл соответствует шаблону - блокируем
            if self.file_guard.matches_pattern(event.dest_path):
                self.file_guard.block_file_rename(event.src_path, event.dest_path,
"переименование")
            # Если исходный файл соответствовал шаблону - тоже блокируем
            elif self.file_guard.matches_pattern(event.src_path):
                self.file_guard.block_file_operation(event.src_path, "переименование")

    def on_modified(self, event):
        # Обработка изменения файла
```

```

        if not event.is_directory and self.file_guard.matches_pattern(event.src_path):
            self.file_guard.block_file_operation(event.src_path, "изменение")

# Основной класс системы защиты файлов
class SimpleFileGuard:
    def __init__(self):
        # Текущая рабочая директория
        self.current_dir = Path.cwd()
        # Файл-шаблон с настройками защиты
        self.template_file = self.current_dir / "template.tbl"
        # Список шаблонов имен файлов для защиты
        self.patterns = []
        # Хеш пароля для доступа
        self.correct_password_hash = None
        # Объект наблюдателя за файловой системой
        self.observer = None
        # Файл блокировки для предотвращения дублирования процессов
        self.lock_file = self.current_dir / "guard.lock"
        # Флаг активности защиты
        self.running = False

        # Настройка системы логирования
        log_file = self.current_dir / 'file_guard.log'
        self.logger = logging.getLogger('FileGuard')
        self.logger.setLevel(logging.INFO)
        self.logger.handlers.clear()

        # Создаем обработчик для записи логов в файл
        file_handler = logging.FileHandler(log_file)
        formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
        file_handler.setFormatter(formatter)
        self.logger.addHandler(file_handler)

    def is_protection_active(self):
        """Проверка активности защиты по состоянию template.tbl"""
        # Пытаемся прочитать файл - если нет прав доступа, значит защита активна
        try:
            with open(self.template_file, 'r') as f:
                f.read(1)
            return False # Файл доступен для чтения - защита не активна
        except PermissionError:
            return True # Доступ запрещен - защита активна
        except:
            return False # Другие ошибки - считаем защиту неактивной

    def cleanup_stale_lock(self):
        """Удаление устаревшего lock-файла если процесс завершился некорректно"""
        try:
            if self.lock_file.exists():

```

```

        os.remove(self.lock_file)
    except:
        pass # Игнорируем ошибки при удалении

def check_prerequisites(self):
    """Проверка всех необходимых условий для работы программы"""
    # Требуем права администратора для работы с ACL
    if not self.is_admin():
        print("Ошибка: Запустите от имени администратора!")
        return False

    # Проверяем наличие файла-шаблона
    if not self.template_file.exists():
        print("Ошибка: Файл template.tbl не найден!")
        return False

    # Если защита уже активна, пропускаем проверку формата
    if self.is_protection_active():
        return True

    # Проверяем корректность формата template.tbl
    return self.validate_template_format()

def is_admin(self):
    """Проверка прав администратора"""
    try:
        return ctypes.windll.shell32.IsUserAnAdmin()
    except:
        return False # В случае ошибки считаем, что прав нет

def validate_template_format(self):
    """Валидация формата и содержимого template.tbl"""
    try:
        # Читаем файл, отбрасывая пустые строки
        with open(self.template_file, 'r') as f:
            lines = [line.strip() for line in f.readlines() if line.strip()]

        # Минимум нужны хеш пароля и хотя бы один шаблон
        if len(lines) < 2:
            print("Ошибка: Нужен пароль и маски файлов")
            return False

        # Проверяем что первая строка - валидный SHA-256 хеш
        password_hash = lines[0]
        if len(password_hash) != 64 or not all(c in '0123456789abcdef' for c in
password_hash):
            print("Ошибка: Хеш пароля должен быть SHA-256")
            return False

        # Сохраняем хеш и шаблоны

```

```

        self.correct_password_hash = password_hash
        self.patterns = lines[1:]
        return True

    except Exception as e:
        print(f"Ошибка чтения template.tbl: {e}")
        return False

    def verify_password(self, password):
        """Проверка введенного пароля против сохраненного хеша"""
        if self.correct_password_hash is None:
            return False
        # Сравниваем хеши
        return hashlib.sha256(password.encode()).hexdigest() == self.correct_password_hash

    def matches_pattern(self, filename):
        """Проверка соответствия имени файла одному из шаблонов"""
        filename = Path(filename).name
        # Используем fnmatch для сравнения с масками типа *.txt
        return any(fnmatch.fnmatch(filename, pattern) for pattern in self.patterns)

    def is_control_file(self, file_path):
        """Определение служебных файлов, которые не нужно защищать"""
        control_files = ["guard.lock", "file_guard.log"]
        return Path(file_path).name in control_files

    def block_file_operation(self, file_path, operation):
        """Блокировка операции с файлом (удаление, изменение)"""
        try:
            # Не блокируем служебные файлы
            if not self.is_control_file(file_path):
                self.protect_single_file(file_path) # Применяем защиту
                self.logger.warning(f"Заблокирована {operation}: {Path(file_path).name}")
        except Exception as e:
            self.logger.error(f"Ошибка блокировки: {e}")

    def block_file_rename(self, src_path, dest_path, operation):
        """Блокировка переименования с восстановлением исходного имени"""
        try:
            # Восстанавливаем исходное имя если целевой файл существует
            if os.path.exists(dest_path):
                os.rename(dest_path, src_path)
            # Защищаем файл если он соответствует шаблону
            if self.matches_pattern(src_path) and not self.is_control_file(src_path):
                self.protect_single_file(src_path)
                self.logger.warning(f"Заблокировано {operation}: {Path(src_path).name} -> {Path(dest_path).name}")
        except Exception as e:
            self.logger.error(f"Ошибка блокировки переименования: {e}")

```

```

def delete_protected_file(self, file_path, operation):
    """Удаление вновь созданного файла, соответствующего шаблону"""
    try:
        # Пытаемся удалить файл с тремя попытками
        for attempt in range(3):
            try:
                if os.path.exists(file_path):
                    os.remove(file_path)
                    self.logger.warning(f"Удален файл: {Path(file_path).name} ({operation})")
                    return True
                return True # Файл уже удален
            except Exception:
                if attempt < 2:
                    time.sleep(0.2) # Ждем перед повторной попыткой
                return False
    except Exception as e:
        self.logger.error(f"Ошибка удаления: {e}")
        return False

def protect_single_file(self, file_path):
    """Установка ACL для запрета записи на конкретный файл"""
    try:
        # Получаем текущие настройки безопасности файла
        sd = win32security.GetFileSecurity(str(file_path),
win32security.DACL_SECURITY_INFORMATION)
        dacl = win32security.ACL()
        everyone_sid = win32security.ConvertStringSidToSid("S-1-1-0") # SID группы
"Все"

        # Запрещаем все операции записи для всех пользователей
        dacl.AddAccessDeniedAce(
            win32security.ACL_REVISION,
            ntsecuritycon.DELETE | ntsecuritycon.WRITE_DAC |
ntsecuritycon.WRITE_OWNER |
            ntsecuritycon.FILE_WRITE_DATA | ntsecuritycon.FILE_WRITE_ATTRIBUTES |
ntsecuritycon.FILE_APPEND_DATA,
            everyone_sid
        )

        # Разрешаем только чтение
        dacl.AddAccessAllowedAce(
            win32security.ACL_REVISION,
            ntsecuritycon.FILE_READ_DATA | ntsecuritycon.FILE_READ_ATTRIBUTES |
ntsecuritycon.FILE_READ_EA | ntsecuritycon.READ_CONTROL,
            everyone_sid
        )

        # Применяем новый ACL
        sd.SetSecurityDescriptorDacl(1, dacl, 0)

```

```

win32security.SetFileSecurity(str(file_path),
win32security.DACL_SECURITY_INFORMATION, sd)
except Exception as e:
    self.logger.error(f"Ошибка защиты файла: {e}")

def unprotect_single_file(self, file_path):
    """Снятие всех ограничений с файла"""
    try:
        sd = win32security.GetFileSecurity(str(file_path),
win32security.DACL_SECURITY_INFORMATION)
        dacl = win32security.ACL()
        everyone_sid = win32security.ConvertStringSidToSid("S-1-1-0")

        # Разрешаем все операции
        dacl.AddAccessAllowedAce(win32security.ACL_REVISION,
win32con.GENERIC_ALL, everyone_sid)
        sd.SetSecurityDescriptorDacl(1, dacl, 0)
        win32security.SetFileSecurity(str(file_path),
win32security.DACL_SECURITY_INFORMATION, sd)
    except Exception as e:
        self.logger.error(f"Ошибка разблокировки: {e}")

def protect_template_file(self):
    """Полная блокировка template.tbl (пустой ACL - доступ запрещен всем)"""
    try:
        sd = win32security.GetFileSecurity(str(self.template_file),
win32security.DACL_SECURITY_INFORMATION)
        dacl = win32security.ACL() # Создаем пустой ACL
        sd.SetSecurityDescriptorDacl(1, dacl, 0) # Устанавливаем пустой DACL
        win32security.SetFileSecurity(str(self.template_file),
win32security.DACL_SECURITY_INFORMATION, sd)
        self.logger.info("template.tbl защищен")
    except Exception as e:
        self.logger.error(f"Ошибка защиты template.tbl: {e}")

def unprotect_template_file(self):
    """Временная разблокировка template.tbl для проверки пароля"""
    try:
        sd = win32security.GetFileSecurity(str(self.template_file),
win32security.DACL_SECURITY_INFORMATION)
        dacl = win32security.ACL()
        everyone_sid = win32security.ConvertStringSidToSid("S-1-1-0")
        # Даем полные права на файл
        dacl.AddAccessAllowedAce(win32security.ACL_REVISION,
win32con.GENERIC_ALL, everyone_sid)
        sd.SetSecurityDescriptorDacl(1, dacl, 0)
        win32security.SetFileSecurity(str(self.template_file),
win32security.DACL_SECURITY_INFORMATION, sd)
        self.logger.info("Защита template.tbl снята")
    except Exception as e:

```



```

        self.logger.error(f"Ошибка снятия защиты: {e}")

def apply_protection(self):
    """Основная функция включения защиты всех файлов"""
    try:
        protected_count = 0
        # Проходим по всем файлам в текущей директории
        for file_path in self.current_dir.iterdir():
            if file_path.is_file() and self.matches_pattern(file_path.name) and not
self.is_control_file(file_path):
                self.protect_single_file(file_path)
                protected_count += 1

        # Блокируем сам template.tbl
        self.protect_template_file()
        # Запускаем мониторинг файловой системы
        self.start_monitoring()

        print(f"Защита активирована. Защищено файлов: {protected_count}")
        self.logger.info(f"Защита активирована. Файлов: {protected_count}")

    except Exception as e:
        print(f"Ошибка применения защиты: {e}")
        self.logger.error(f"Ошибка применения защиты: {e}")

def remove_protection(self):
    """Снятие защиты со всех файлов"""
    try:
        # Останавливаем мониторинг
        self.stop_monitoring()

        unprotected_count = 0
        # Разблокируем все файлы в директории
        for file_path in self.current_dir.iterdir():
            if file_path.is_file():
                try:
                    self.unprotect_single_file(file_path)
                    unprotected_count += 1
                except:
                    pass # Пропускаем файлы, которые не удалось разблокировать

        # Разблокируем template.tbl
        self.unprotect_template_file()

        print(f"Защита отключена. Разблокировано файлов: {unprotected_count}")
        self.logger.info(f"Защита отключена. Файлов: {unprotected_count}")

    except Exception as e:
        print(f"Ошибка снятия защиты: {e}")
        self.logger.error(f"Ошибка снятия защиты: {e}")

```

```

def start_monitoring(self):
    """Запуск наблюдателя за файловой системой"""
    try:
        event_handler = FileGuardEventHandler(self)
        self.observer = Observer()
        # Наблюдаем только за текущей директорией (без рекурсии)
        self.observer.schedule(event_handler, str(self.current_dir), recursive=False)
        self.observer.start()
        self.logger.info("Мониторинг запущен")
    except Exception as e:
        self.logger.error(f"Ошибка запуска мониторинга: {e}")

def stop_monitoring(self):
    """Остановка наблюдателя"""
    try:
        if self.observer:
            self.observer.stop()
            self.observer.join(timeout=3) # Ждем завершения потока
            self.observer = None
            self.logger.info("Мониторинг остановлен")
    except Exception as e:
        self.logger.error(f"Ошибка остановки мониторинга: {e}")

def hide_console(self):
    """Скрытие окна консоли после запуска защиты"""
    try:
        window = win32gui.GetForegroundWindow()
        win32gui.ShowWindow(window, wcon.SW_HIDE)
    except Exception as e:
        self.logger.error(f"Ошибка скрытия консоли: {e}")

def create_lock_file(self):
    """Создание lock-файла для предотвращения дублирующих процессов"""
    try:
        with open(self.lock_file, 'w') as f:
            f.write(str(os.getpid())) # Сохраняем PID текущего процесса
        self.logger.info(f"Lock-файл создан. PID: {os.getpid()}")
    except Exception as e:
        self.logger.error(f"Ошибка создания lock-файла: {e}")

def get_guard_pid(self):
    """Получение PID процесса защиты из lock-файла"""
    try:
        if self.lock_file.exists():
            with open(self.lock_file, 'r') as f:
                return int(f.read().strip())
    except:
        pass
    return None

```

```

def terminate_guard_process(self):
    """Принудительное завершение процесса защиты"""
    pid = self.get_guard_pid()
    if not pid:
        return False # PID не найден

    try:
        # Используем taskkill для гарантированного завершения
        os.system(f'taskkill /pid {pid} /f >nul 2>&1')
        self.logger.info(f'Процесс защиты {pid} завершен')
        return True
    except:
        return False

def run_protection(self):
    """Основной рабочий цикл защиты, работает в фоновом режиме"""
    try:
        # Создаем lock-файл чтобы отметить запущенный процесс
        self.create_lock_file()

        # Скрываем окно консоли
        self.hide_console()

        # Применяем защиту ко всем файлам
        self.apply_protection()

        self.logger.info("Защита запущена в скрытом режиме")
        print("Защита запущена. Консоль скрыта.")

        # Бесконечный цикл ожидания
        while True:
            time.sleep(1) # Минимальная нагрузка на CPU

    except KeyboardInterrupt:
        # Обработка Ctrl+C (в теории не должно происходить в скрытом режиме)
        self.logger.info("Получен сигнал KeyboardInterrupt")
    except Exception as e:
        self.logger.error(f"Ошибка в работе защиты: {e}")
    finally:
        # Гарантированная очистка при завершении
        self.remove_protection() # Снимаем защиту
        if self.lock_file.exists():
            os.remove(self.lock_file) # Удаляем lock-файл
            self.logger.info("Защита корректно остановлена")

def start_protection(self):
    """Запуск защиты с проверкой пароля"""
    # Даем 3 попытки ввода пароля
    for attempt in range(3):

```

```

password = input("Введите пароль для включения защиты: ")
if self.verify_password(password):
    print("Пароль верный. Запуск защиты...")
    # Запускаем основной цикл
    self.run_protection()
    return True
else:
    remaining = 2 - attempt
    print(f"Неверный пароль. Осталось попыток: {remaining}")

print("Превышено количество попыток.")
return False

def stop_protection(self):
    """Остановка защиты с обязательной проверкой пароля"""
    # Проверяем что защита вообще активна
    if not self.is_protection_active():
        print("Защита не активна!")
        return True

    print("\n" + "=" * 50)
    print("ОСТАНОВКА ЗАЩИТЫ")
    print("=" * 50)

    # Временно разблокируем template.tbl для чтения пароля
    self.unprotect_template_file()

    # Перечитываем настройки из файла
    if not self.validate_template_format():
        print("Ошибка: Неверный формат файла template.tbl!")
        return False

    # Сразу блокируем файл обратно
    self.protect_template_file()

    # Проверяем пароль
    for attempt in range(3):
        password = input("Введите пароль для отключения защиты: ")
        if self.verify_password(password):
            print("Пароль верный. Останавливаем защиту...")

            # Пытаемся завершить процесс защиты
            if self.terminate_guard_process():
                print("Процесс защиты завершен")
            else:
                print("Не удалось завершить процесс автоматически")

            # Небольшая пауза для гарантии завершения
            time.sleep(1)

```

```

# Дублирующее снятие защиты на случай проблем с завершением процесса
print("Снимаю защиту с файлов...")
self.remove_protection()

# Очищаем lock-файл
if self.lock_file.exists():
    os.remove(self.lock_file)

print("Защита успешно остановлена!")
return True
else:
    remaining = 2 - attempt
    print(f"Неверный пароль. Осталось попыток: {remaining}")

print("Превышено количество попыток. Защита продолжает работу.")
return False

def show_status(self):
    """Отображение текущего статуса защиты"""
    status = "ВКЛЮЧЕНА" if self.is_protection_active() else "ВЫКЛЮЧЕНА"
    pid = self.get_guard_pid()

    print(f"Статус защиты: {status}")
    if pid:
        print(f"PID процесса защиты: {pid}")
        print(f"Текущая папка: {self.current_dir}")

def main():
    """Главная функция - точка входа в программу"""
    guard = SimpleFileGuard()

    # Проверяем условия перед выполнением
    if not guard.check_prerequisites():
        return # Выходим если условия не выполнены

    # Обработка аргументов командной строки
    if len(sys.argv) > 1:
        if sys.argv[1] == "start":
            # Запуск защиты
            if not guard.is_protection_active():
                guard.start_protection()
            else:
                print("Защита уже активна!")

        elif sys.argv[1] == "stop":
            # Остановка защиты
            guard.stop_protection()

        elif sys.argv[1] == "status":

```

```

        # Показать статус
        guard.show_status()

    else:
        print("Неизвестная команда")
else:
    # Вывод справки если нет аргументов
    print("FileGuard - Защита файлов")
    print("=" * 50)
    print("Использование: python file_guard.py [start|stop|status]")
    print("")
    print("Команды:")
    print(" start - запуск защиты (консоль скроется)")
    print(" stop  - остановка защиты с проверкой пароля")
    print(" status - статус защиты")

if __name__ == "__main__":
    main()

```

Примеры работы программы

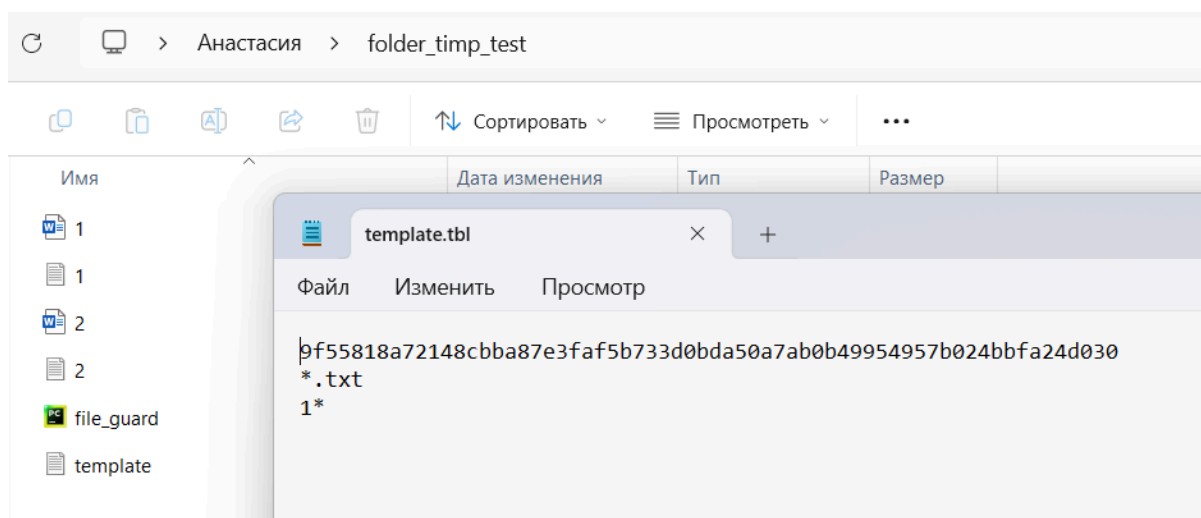


Рисунок 1 – Рабочая папка и содержимое файла template.tbl

```
Administrator: Командная строка - python file_guard.py start
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.
Active code page: 65001

C:\Windows\System32>cd C:\Users\Анастасия\folder_timp_test

C:\Users\Анастасия\folder_timp_test>python file_guard.py
FileGuard - Защита файлов
=====
Использование: python file_guard.py [start|stop|status]

Команды:
  start - запуск защиты (консоль скроется)
  stop  - остановка защиты с проверкой пароля
  status - статус защиты

C:\Users\Анастасия\folder_timp_test>python file_guard.py start
Введите пароль для включения защиты:
```

Рисунок 2 – Процесс запуска защиты

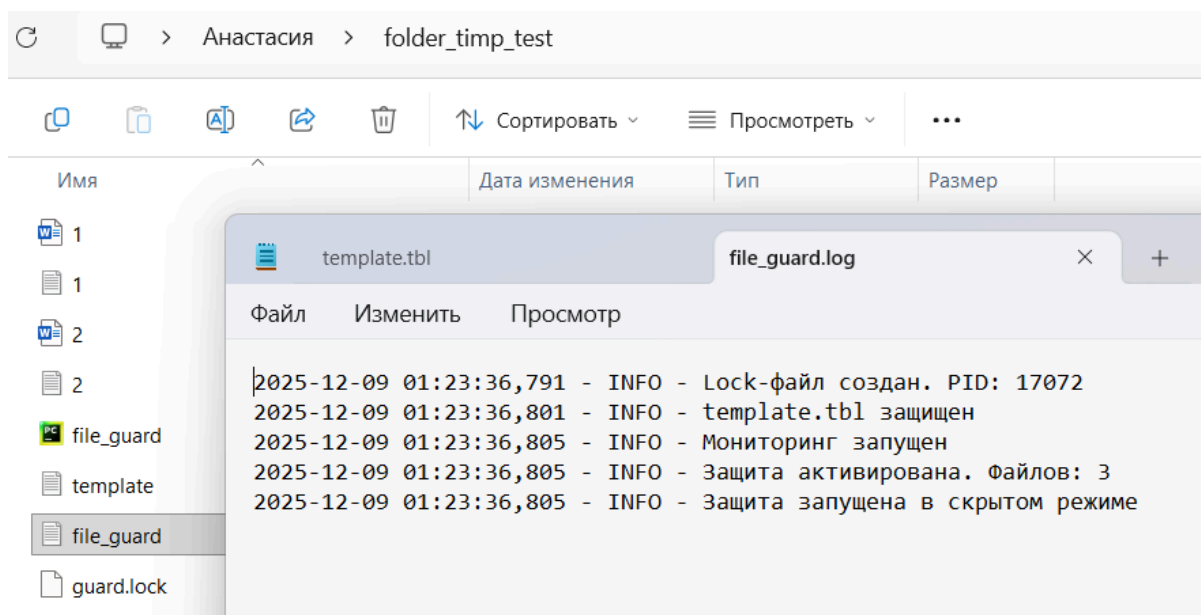


Рисунок 3 – Содержимое рабочей папки после запуска защиты

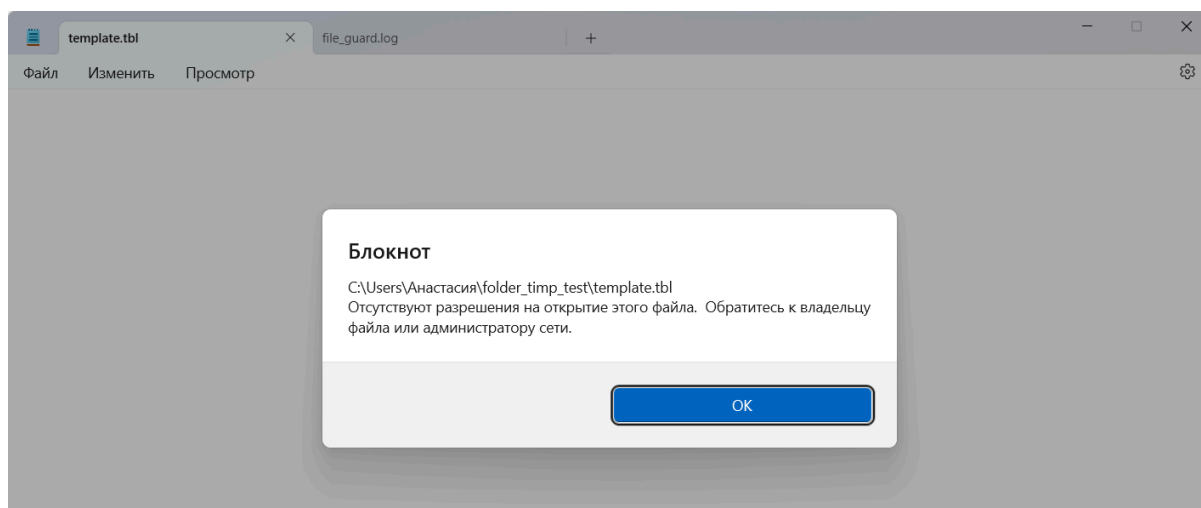


Рисунок 4 – Попытка открытия файла template.tbl после запуска защиты

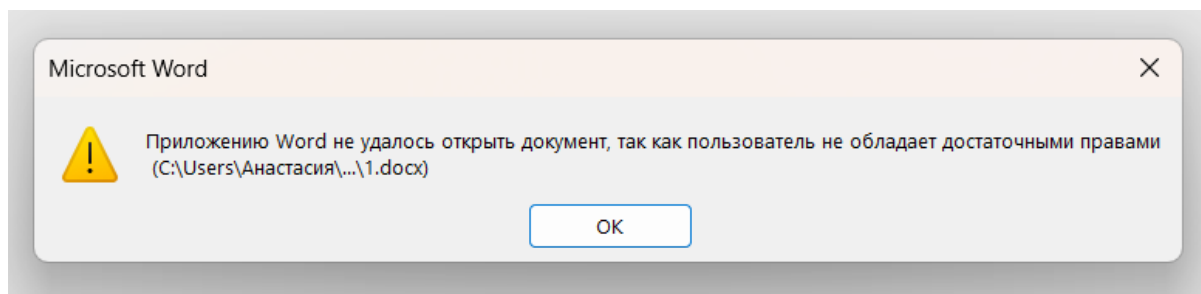


Рисунок 5 – Попытка открытия файла 1.docx после запуска защиты

Блокнот

C:\Users\Анастасия\folder_timp_test\1.txt

Отсутствуют разрешения на открытие этого файла. Обратитесь к владельцу файла или администратору сети.

OK

Рисунок 6 – Попытка открытия файла 1.txt после запуска защиты

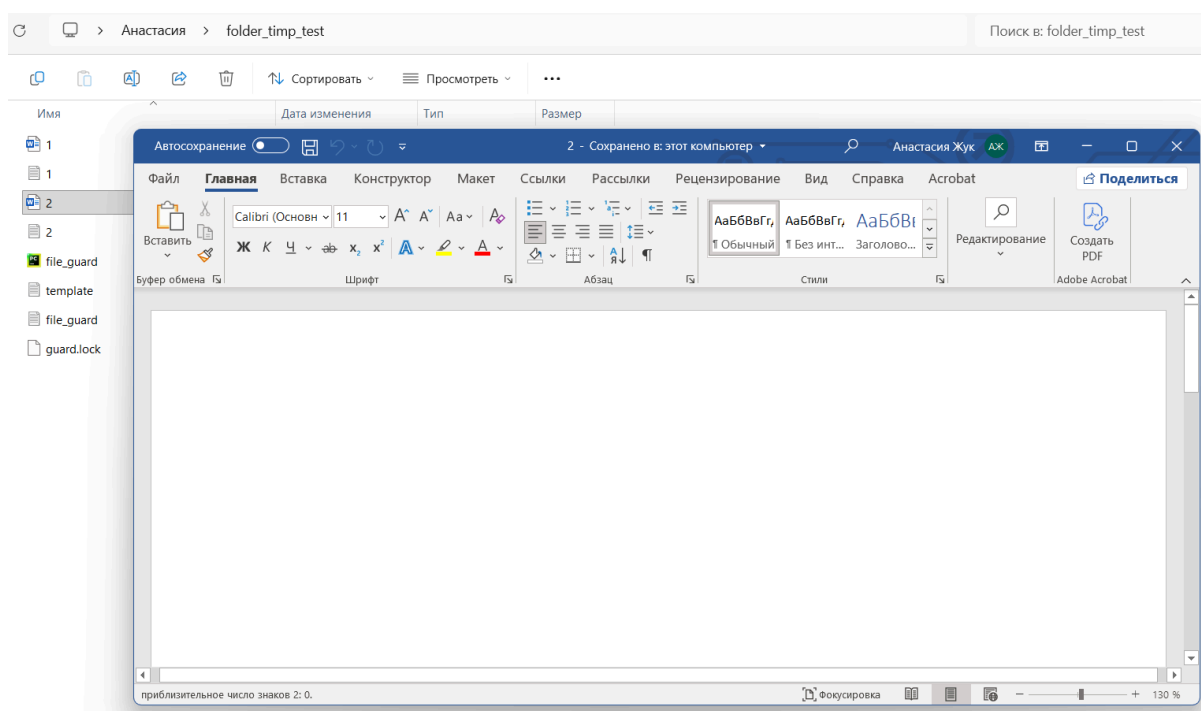


Рисунок 7 – Открытие файла 2.docx после запуска защиты

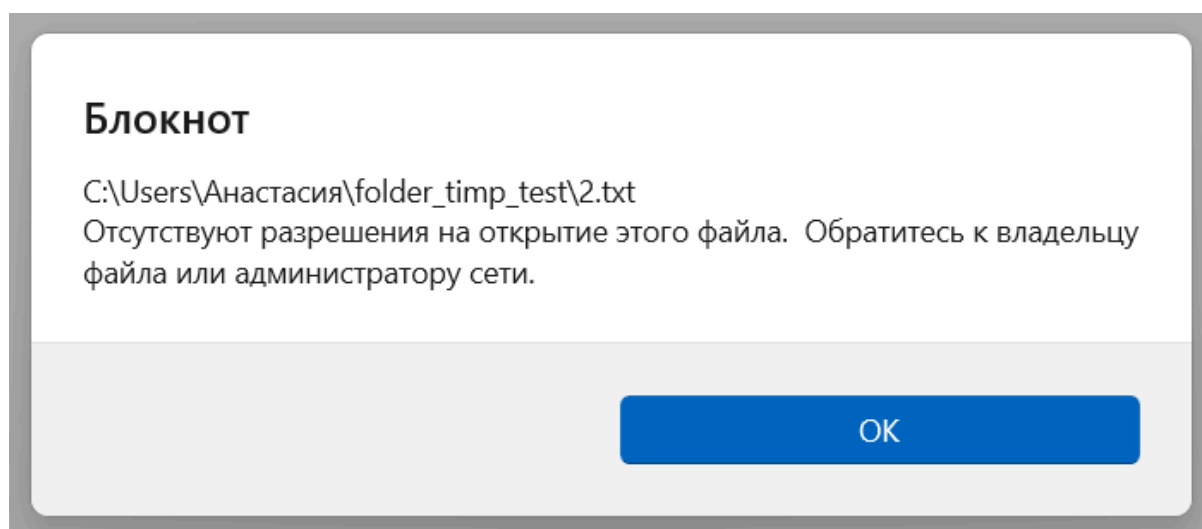


Рисунок 8 – Попытка открытия файла 2.txt после запуска защиты

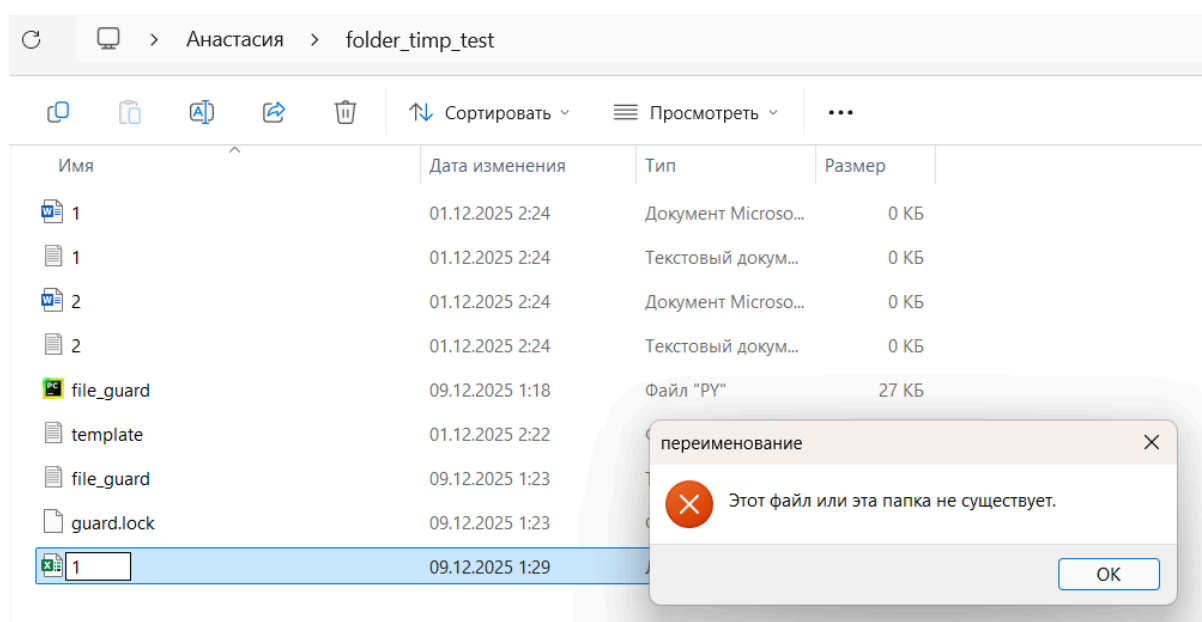


Рисунок 9 – Попытка создания файла с запрещенным именем

Имя	Дата изменения	Тип	Размер
1	01.12.2025 2:24	Документ Microso...	0 КБ
1	01.12.2025 2:24	Текстовый докум...	0 КБ
2	01.12.2025 2:24	Документ Microso...	0 КБ
2	01.12.2025 2:24	Текстовый докум...	0 КБ
file_guard	09.12.2025 1:18	Файл "PY"	27 КБ
template	01.12.2025 2:22	Файл "TBL"	1 КБ
file_guard	09.12.2025 1:23	Текстовый докум...	1 КБ
guard.lock	09.12.2025 1:23	Файл "LOCK"	1 КБ
Лист Microsoft Excel	09.12.2025 1:29	Лист Microsoft Ex...	7 КБ

Рисунок 10 – Возможность создавать файлы, не запрещенные файлом template.tbl

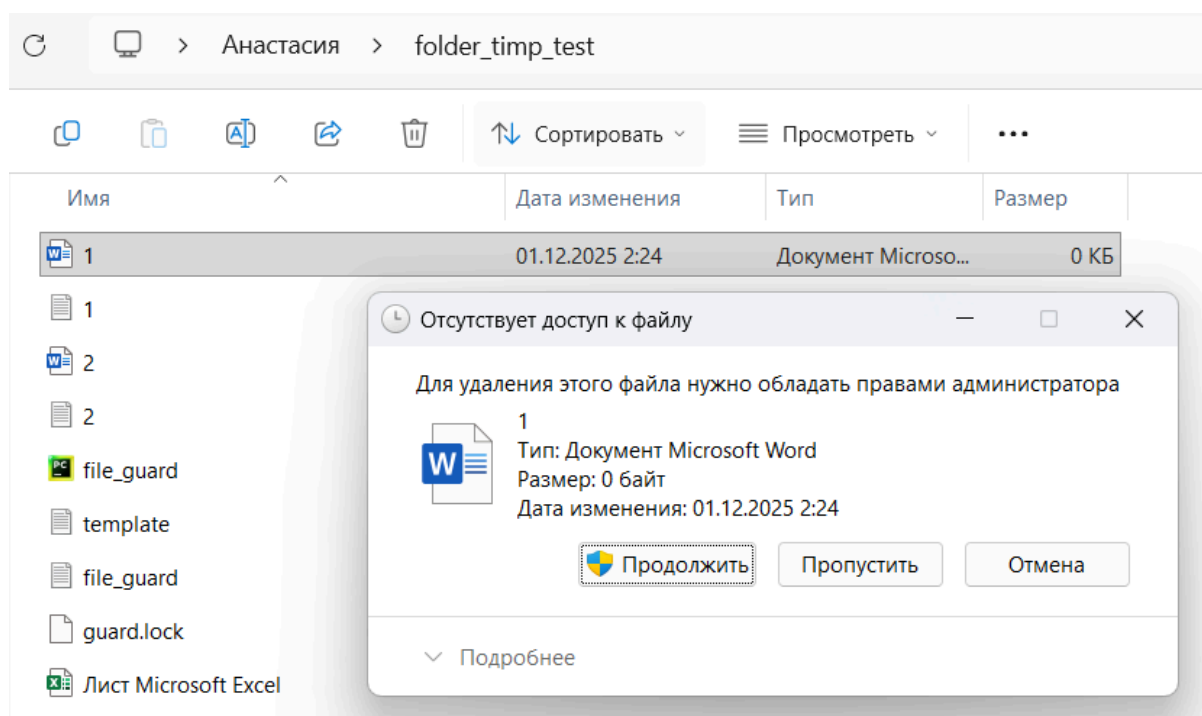


Рисунок 11 – Попытка удаления файла, соответствующего маске из файла template.tbl

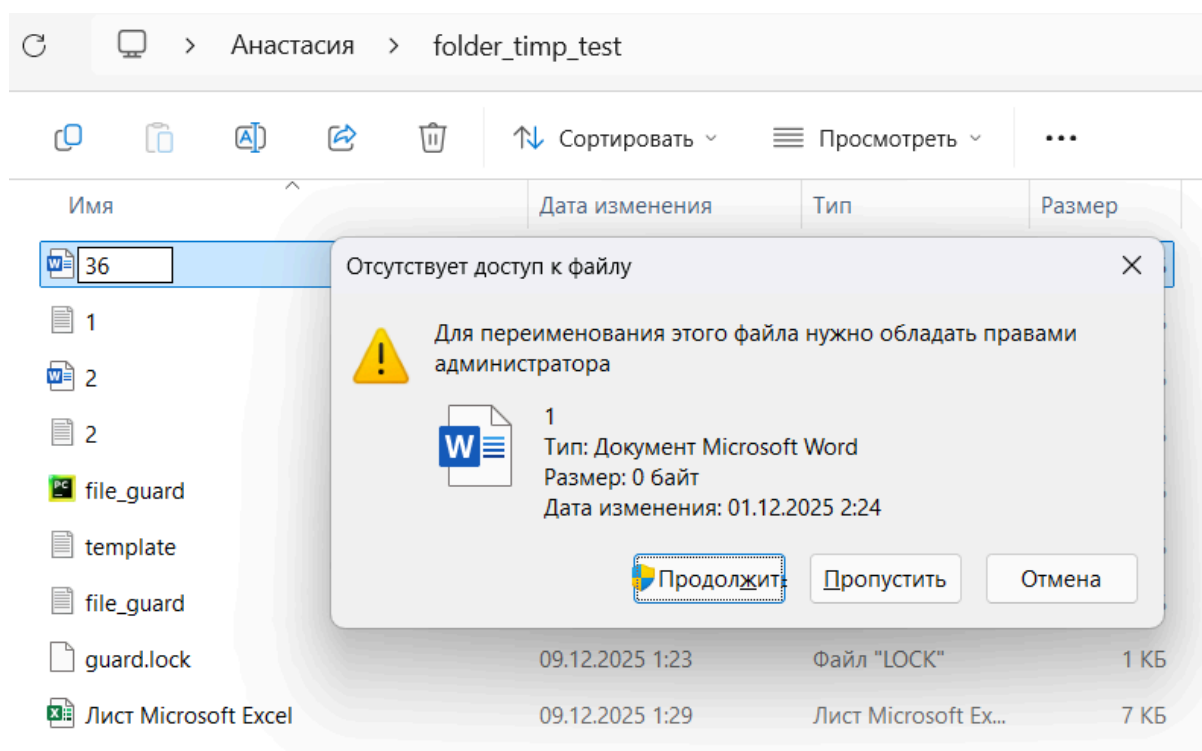


Рисунок 12 – Попытка переименования файла, соответствующего маске из файла template.tbl

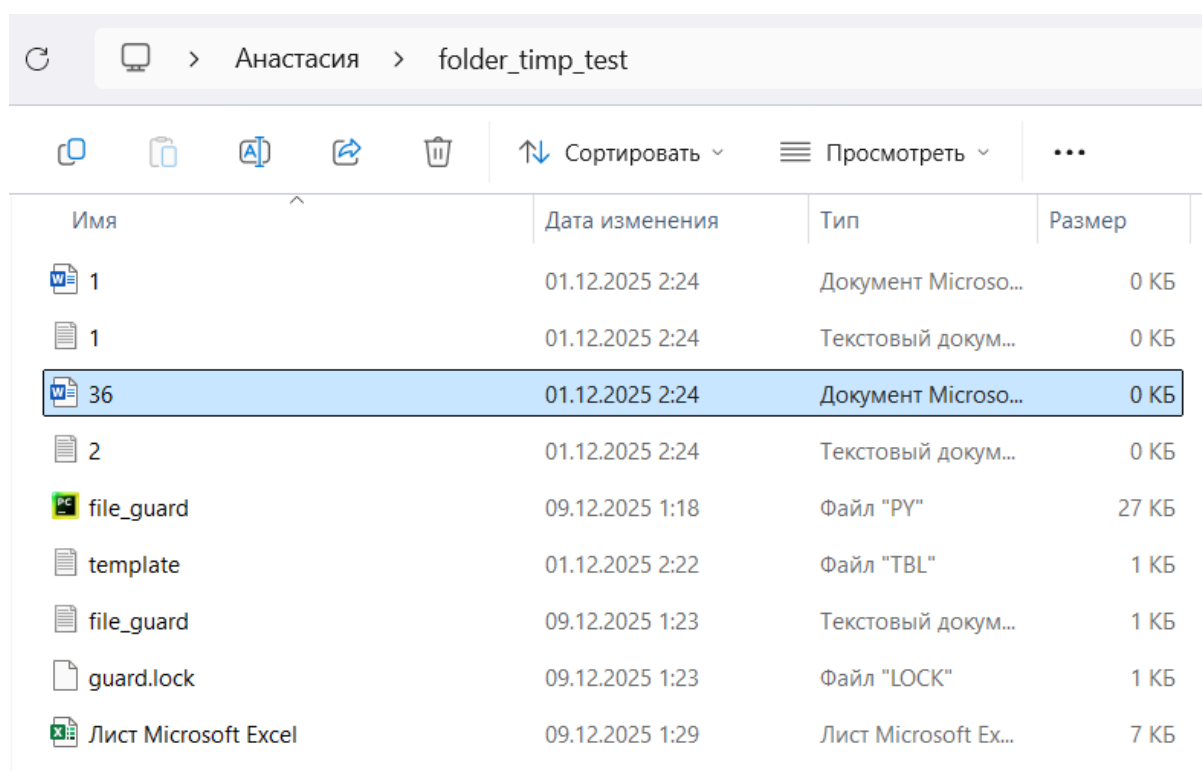


Рисунок 13 – Возможность переименования файла, не соответствующего маске из файла template.tbl

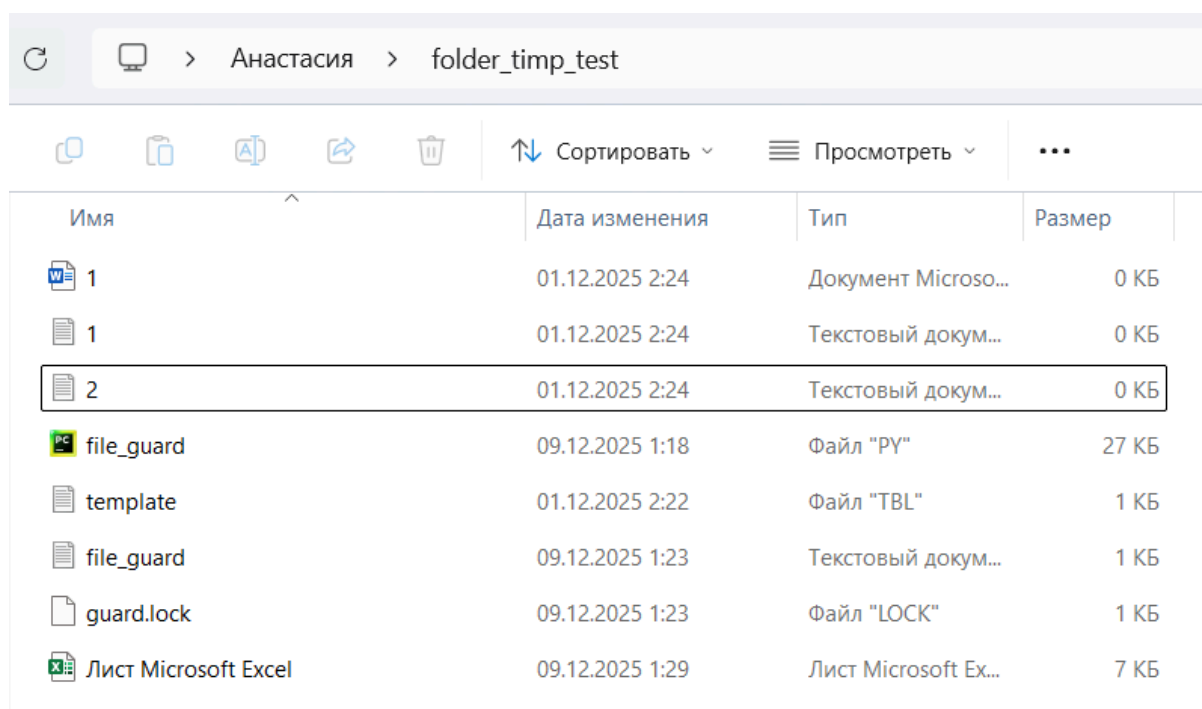


Рисунок 14 – Возможность удаления файла, не соответствующего маске из файла template.tbl

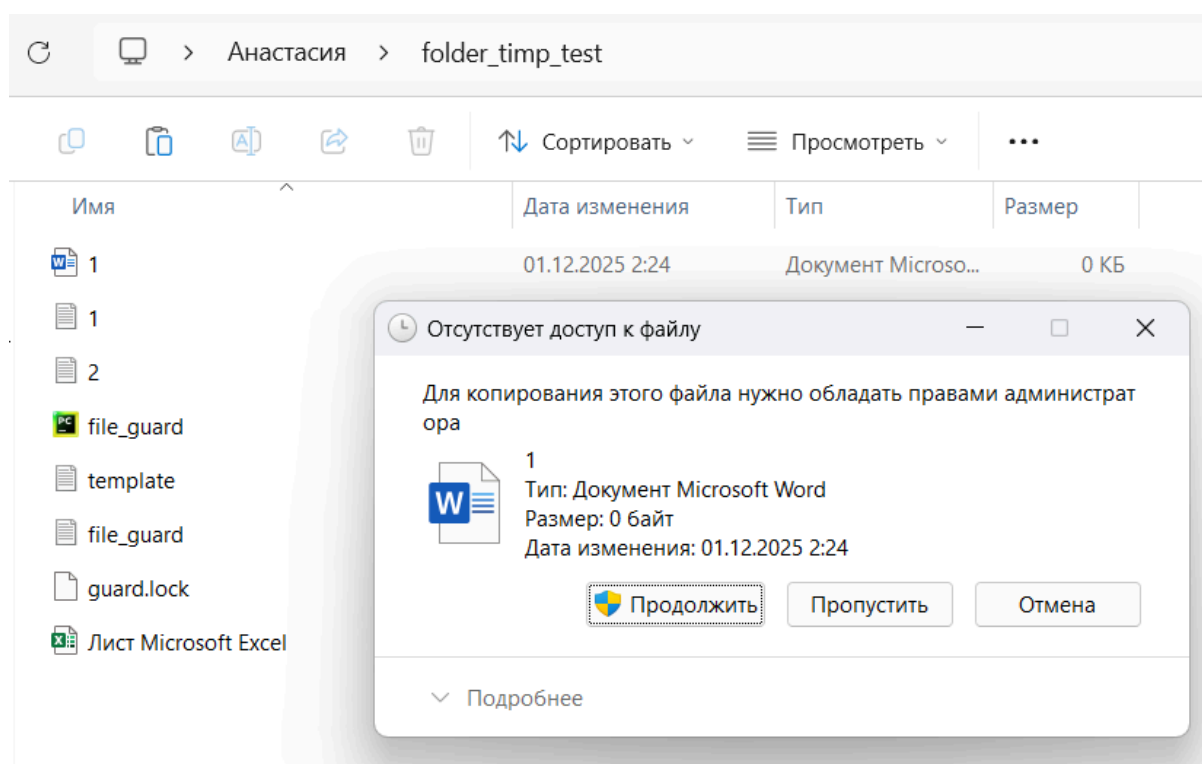


Рисунок 15 – Запрет на копирование файла, соответствующего маске из файла template.tbl

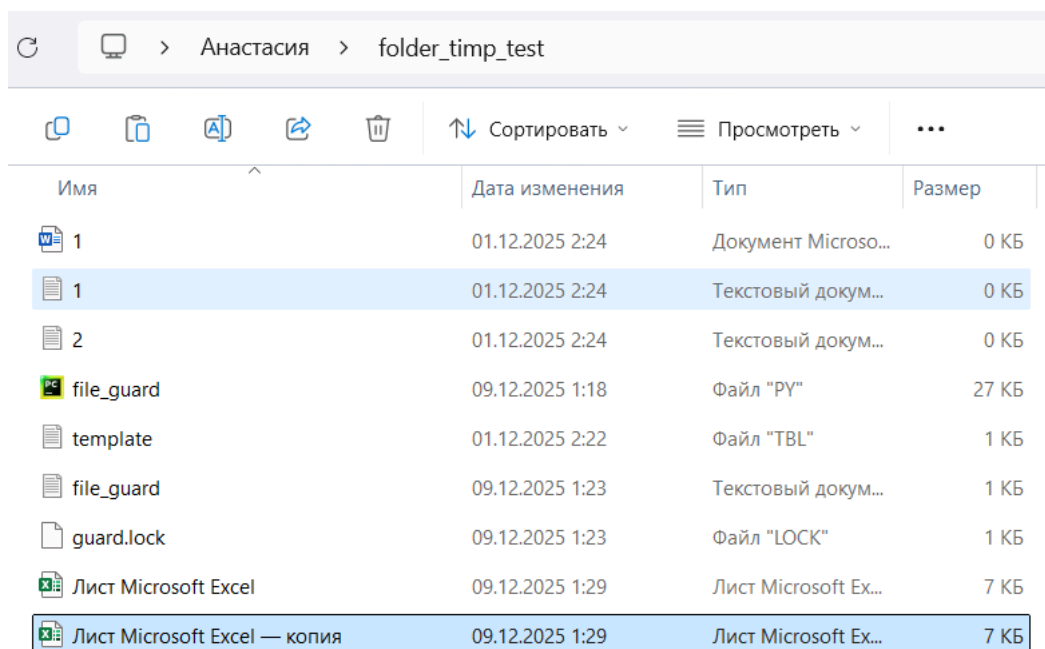


Рисунок 16 – Возможность копирования файла, не соответствующего маске из файла template.tbl

```
Administrator: Командная строка
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.
Active code page: 65001

C:\Windows\System32>cd C:\Users\Анастасия\folder_timp_test

C:\Users\Анастасия\folder_timp_test>python file_guard.py status
Статус защиты: ВКЛЮЧЕНА
PID процесса защиты: 17072
Текущая папка: C:\Users\Анастасия\folder_timp_test

C:\Users\Анастасия\folder_timp_test>python file_guard.py stop

=====
ОСТАНОВКА ЗАЩИТЫ
=====
Введите пароль для отключения защиты: nastya
Пароль верный. Останавливаем защиту...
Active code page: 65001
Процесс защиты завершен
Снимаю защиту с файлов...
Защита отключена. Разблокировано файлов: 9
Защита успешно остановлена!

C:\Users\Анастасия\folder_timp_test>
```

Рисунок 17 – Снятие защиты

Задание 1(б). Веб-скрипт

Техническое задание

Реализовать JavaScript код, внедряемый в код HTML-документа, который реализует защиту от копирования в буфер (нет возможности выделять содержимое, копировать в буфер как текст или через скриншот и т.д.) и сохранения всех страниц, вызываемых с текущей. При этом печать этих страниц на бумажный носитель должна быть доступна. Отключение скрипта должно происходить с использованием пароля, хранящегося в теле скрипта в зашифрованном виде.

Описание продукта, его функционал

Продукт представляет собой одностраничный веб-сайт с многостраничной навигацией, в который интегрирована система защиты от несанкционированного копирования контента. Основной функционал включает навигацию между разделами без перезагрузки страницы, реализованную средствами JavaScript, что обеспечивает плавный и быстрый переход между разделами («Главная», «О нас», «Услуги» и «Контакты»).

В целях безопасности полностью отключено выделение текста как с помощью мыши, так и с клавиатуры, что исключает возможность копирования содержимого стандартными способами. Дополнительно блокируется контекстное меню, вызываемое правой кнопкой мыши, а также комбинации клавиш, используемые для копирования и доступа к инструментам разработчика: Ctrl+C (копирование), Ctrl+U (просмотр исходного кода), Ctrl+S (сохранение страницы), Ctrl+X (вырезание), F12 (открытие консоли разработчика) и PrintScreen (создание скриншота). При этом возможность печати содержимого на бумажный носитель сохраняется (Ctrl+P) – пользователь может вывести любую страницу на принтер без ограничений.

Система защиты поддерживает парольное отключение: пароль хранится в зашифрованном формате base64, что обеспечивает базовый уровень безопасности. Для деактивации защиты предусмотрено модальное окно, расположенное в левом нижнем углу, которое запрашивает ввод пароля. После успешной проверки все ограничения снимаются, и функционал копирования становится доступным.

Инструкция по применению

После открытия файла timp_1b.html в браузере сайт автоматически загружается, и система защиты активируется сразу после завершения загрузки страницы. При этом обеспечивается защита от несанкционированного контента, описанная в разделе «Описание продукта, его функционал». Можно убедиться, что она корректно работает, протестировав все ограничения, описанные ранее. Важно отметить, что PrintScreen все таки будет работать, так как это системная клавиша, которая обрабатывается операционной системой на уровне всего экрана, а не отдельного окна браузера. Браузер не может перехватить это событие, так как оно не генерирует стандартное событие клавиатуры (keydown/keyup) в контексте веб-страницы.

Для отключения защиты требуется ввести пароль. В левом нижнем углу можно увидеть модальное окно. В него необходимо ввести установленный пароль (по умолчанию используется nastyа) и нажать кнопку «Разблокировать». Если пароль введен верно, то защита полностью отключается: восстанавливается возможность выделения текста, работа контекстного меню и всех стандартных сочетаний клавиш для копирования (также можно протестировать все клавиши и их комбинации). После этого содержимое страницы становится доступным для копирования и сохранения обычными способами.

HTML код с внедренным JavaScript кодом

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Многостраничный сайт</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: 'Segoe UI', sans-serif;
    }

    body {
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
      color: white;
      padding: 20px;
    }

    .container {
      max-width: 1200px;
      margin: 0 auto;
    }

    header {
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 20px 0;
      border-bottom: 2px solid rgba(255,255,255,0.1);
    }

    .logo {
      font-size: 28px;
      font-weight: bold;
    }

    nav {
      display: flex;
      gap: 20px;
    }

    .nav-link {
      color: white;
      text-decoration: none;
      padding: 10px 20px;
```

```

    border-radius: 8px;
    transition: all 0.3s;
    cursor: pointer;
}

.nav-link:hover, .nav-link.active {
    background: rgba(255,255,255,0.2);
}

.page {
    display: none;
    padding: 40px 0;
    animation: fadeIn 0.5s ease;
}

.page.active {
    display: block;
}

h1 {
    font-size: 48px;
    margin-bottom: 20px;
    text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}

h2 {
    font-size: 32px;
    margin-bottom: 15px;
}

p {
    font-size: 18px;
    line-height: 1.6;
    margin-bottom: 20px;
    max-width: 800px;
}

.content {
    background: rgba(255,255,255,0.1);
    backdrop-filter: blur(10px);
    border-radius: 15px;
    padding: 30px;
    margin-top: 30px;
    border: 1px solid rgba(255,255,255,0.2);
}

.cards {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 20px;

```

```

    margin-top: 30px;
  }

  .card {
    background: rgba(255,255,255,0.15);
    padding: 25px;
    border-radius: 10px;
    transition: transform 0.3s;
  }

  .card:hover {
    transform: translateY(-5px);
  }

  footer {
    text-align: center;
    padding: 30px 0;
    margin-top: 40px;
    border-top: 2px solid rgba(255,255,255,0.1);
  }

  @keyframes fadeIn {
    from { opacity: 0; transform: translateY(20px); }
    to { opacity: 1; transform: translateY(0); }
  }

  @media (max-width: 768px) {
    header {
      flex-direction: column;
      gap: 15px;
    }

    nav {
      flex-wrap: wrap;
      justify-content: center;
    }

    h1 {
      font-size: 36px;
    }
  }
</style>
</head>
<body>
  <div class="container">
    <header>
      <div class="logo">Мой сайт</div>
      <nav>
        <a class="nav-link active" data-page="home">Главная</a>
        <a class="nav-link" data-page="about">О нас</a>

```

```

    <a class="nav-link" data-page="services">Услуги</a>
    <a class="nav-link" data-page="contact">Контакты</a>
  </nav>
</header>

<main>
  <!-- Главная страница -->
  <section id="home" class="page active">
    <h1>Добро пожаловать!</h1>
    <p>Это компактный многостраничный сайт, созданный одним HTML-файлом.
    Используйте навигацию для перехода между страницами.</p>

    <div class="content">
      <h2>Преимущества</h2>
      <div class="cards">
        <div class="card">
          <h3>Быстро</h3>
          <p>Загрузка без перезагрузки страницы</p>
        </div>
        <div class="card">
          <h3>Адаптивно</h3>
          <p>Работает на всех устройствах</p>
        </div>
        <div class="card">
          <h3>Просто</h3>
          <p>Весь код в одном файле</p>
        </div>
      </div>
    </div>
  </section>

  <!-- О нас -->
  <section id="about" class="page">
    <h1>О нашей компании</h1>
    <p>Мы создаем инновационные решения для вашего бизнеса.</p>

    <div class="content">
      <h2>Наша команда</h2>
      <div class="cards">
        <div class="card">
          <h3>Разработчики</h3>
          <p>Профессионалы с опытом от 5 лет</p>
        </div>
        <div class="card">
          <h3>Дизайнеры</h3>
          <p>Создают современные интерфейсы</p>
        </div>
        <div class="card">
          <h3>Аналитики</h3>
          <p>Помогают достигать бизнес-целей</p>
        </div>
      </div>
    </div>
  </section>

```

```

        </div>
    </div>
</div>
</section>

<!-- Услуги -->
<section id="services" class="page">
    <h1>Наши услуги</h1>
    <p>Предлагаем комплексные решения для вашего успеха.</p>

    <div class="content">
        <div class="cards">
            <div class="card">
                <h3>Веб-разработка</h3>
                <p>Создание сайтов и веб-приложений</p>
            </div>
            <div class="card">
                <h3>Мобильные приложения</h3>
                <p>Разработка для iOS и Android</p>
            </div>
            <div class="card">
                <h3>Облачные решения</h3>
                <p>Масштабируемая инфраструктура</p>
            </div>
            <div class="card">
                <h3>Консалтинг</h3>
                <p>Стратегия цифровой трансформации</p>
            </div>
        </div>
    </div>
</section>

<!-- Контакты -->
<section id="contact" class="page">
    <h1>Свяжитесь с нами</h1>
    <p>Мы всегда готовы ответить на ваши вопросы.</p>

    <div class="content">
        <h2>Контактная информация</h2>
        <div class="cards">
            <div class="card">
                <h3>Email</h3>
                <p>info@mysite.ru</p>
            </div>
            <div class="card">
                <h3>Телефон</h3>
                <p>+7 (999) 123-45-67</p>
            </div>
            <div class="card">
                <h3>Адрес</h3>

```

```

        <p>Москва, ул. Примерная, 123</p>
    </div>
</div>
</div>
</section>
</main>
</div>

<script>
    // Навигация между страницами
    document.querySelectorAll('.nav-link').forEach(link => {
        link.addEventListener('click', function(e) {
            e.preventDefault();

            // Получаем ID страницы
            const pageId = this.getAttribute('data-page');

            // Скрываем все страницы
            document.querySelectorAll('.page').forEach(page => {
                page.classList.remove('active');
            });

            // Показываем выбранную страницу
            document.getElementById(pageId).classList.add('active');

            // Обновляем активную ссылку в навигации
            document.querySelectorAll('.nav-link').forEach(link => {
                link.classList.remove('active');
            });
            this.classList.add('active');

            // Прокручиваем наверх страницы
            window.scrollTo({ top: 0, behavior: 'smooth' });
        });
    });

    // Обработка URL (чтобы можно было делиться ссылками)
    function navigateToPage(pageId) {
        document.querySelectorAll('.page').forEach(page => {
            page.classList.remove('active');
        });
        document.getElementById(pageId).classList.add('active');

        document.querySelectorAll('.nav-link').forEach(link => {
            link.classList.remove('active');
            if(link.getAttribute('data-page') === pageId) {
                link.classList.add('active');
            }
        });
    }

```

```

        history.pushState({}, "", `#${pageId}`);
    }

    // Обработка хэша в URL при загрузке
    window.addEventListener('load', () => {
        const hash = window.location.hash.substring(1);
        if (hash && document.getElementById(hash)) {
            navigateToPage(hash);
        }
    });

    // Обработка кнопок браузера "назад/вперед"
    window.addEventListener('popstate', () => {
        const hash = window.location.hash.substring(1);
        if (hash && document.getElementById(hash)) {
            navigateToPage(hash);
        }
    });
</script>

<!-- Модальное окно для ввода пароля -->
<div id="passwordModal" class="password-modal">
    <div class="password-form">
        <h3>Введите пароль для отключения защиты</h3>
        <input type="password" id="passwordInput" class="password-input"
placeholder="Пароль">
        <button    onclick="checkPassword()"
class="password-btn">Разблокировать</button>
        <p id="errorMsg" style="color: red; display: none;">Неверный пароль!</p>
    </div>
</div>

<script>
    // Зашифрованный пароль (base64 encoded)
    const encryptedPassword = 'cGFzc3dvcmQxMjM='; // password123 в base64

    // Функция для отображения модального окна
    function showPasswordModal() {
        document.getElementById('passwordModal').style.display = 'block';
    }

    // Функция проверки пароля
    function checkPassword() {
        const input = document.getElementById('passwordInput').value;
        const decodedPassword = atob(encryptedPassword);

        if (input === decodedPassword) {
            disableProtection();
            document.getElementById('passwordModal').style.display = 'none';
        } else {

```



```

        document.getElementById('errorMsg').style.display = 'block';
    }
}

// Функция отключения защиты
function disableProtection() {
    // Восстанавливаем выделение текста
    document.body.style.userSelect = 'text';
    document.body.style.webkitUserSelect = 'text';
    document.body.style.MozUserSelect = 'text';

    // Восстанавливаем контекстное меню
    document.oncontextmenu = null;

    // Удаляем обработчики событий
    document.removeEventListener('keydown', preventKeyCombinations);
    document.removeEventListener('mousedown', preventRightClick);
    document.removeEventListener('selectstart', preventSelection);

    alert('Защита отключена!');
}

// Функции для предотвращения действий
function preventSelection(e) {
    e.preventDefault();
    return false;
}

function preventRightClick(e) {
    if (e.button === 2) {
        e.preventDefault();
        return false;
    }
}

function preventKeyCombinations(e) {
    // Блокировка Ctrl+C, Ctrl+U, F12, PrintScreen и др.
    if (e.ctrlKey && (e.key === 'c' || e.key === 'u' || e.key === 's' || e.key === 'x')) {
        e.preventDefault();
        return false;
    }

    if (e.key === 'F12' || e.key === 'PrintScreen') {
        e.preventDefault();
        return false;
    }
}

// Инициализация защиты
function initProtection() {

```

```

// Запрещаем выделение текста
document.body.style.userSelect = 'none';
document.body.style.webkitUserSelect = 'none';
document.body.style.MozUserSelect = 'none';

// Запрещаем контекстное меню
document.oncontextmenu = function() { return false; };

// Добавляем обработчики событий
document.addEventListener('selectstart', preventSelection);
document.addEventListener('mousedown', preventRightClick);
document.addEventListener('keydown', preventKeyCombinations);

// Защита от скриншотов (частичная)
document.body.style.webkitTouchCallout = 'none';
document.body.style.webkitUserSelect = 'none';
}

// Инициализация при загрузке страницы
window.onload = function() {
    initProtection();
};
</script>
</body>
</html>

```

Примеры работы программы

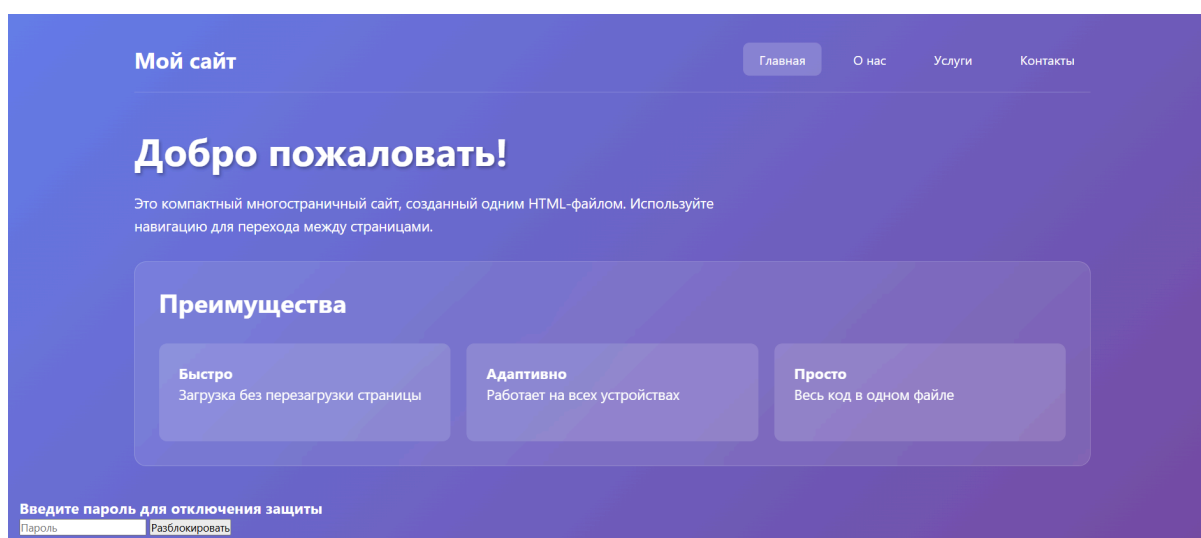


Рисунок 18 – Обзор раздела «Главная» в режиме защиты

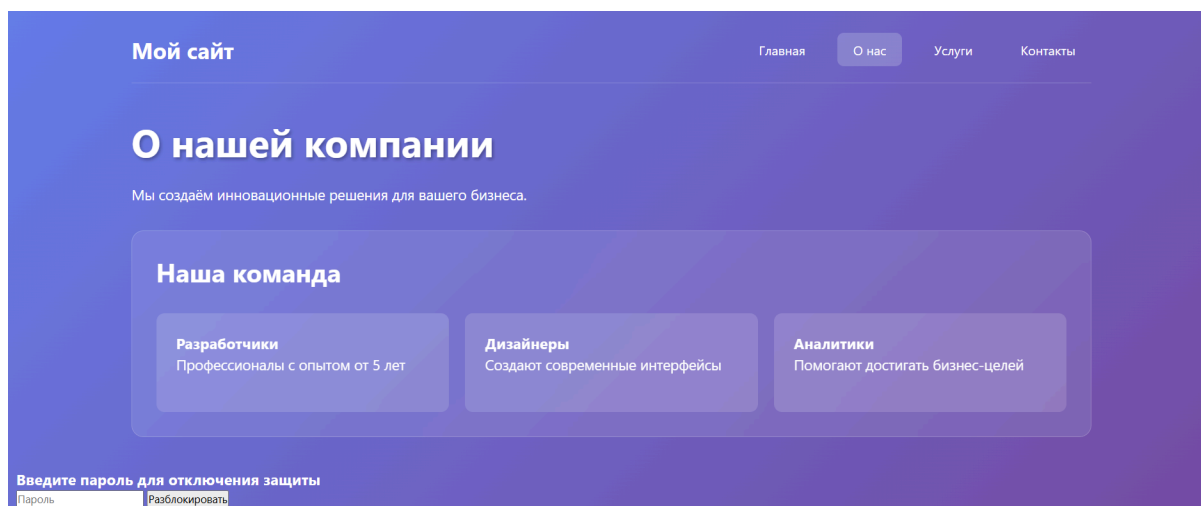


Рисунок 19 – Обзор раздела «О нас» в режиме защиты

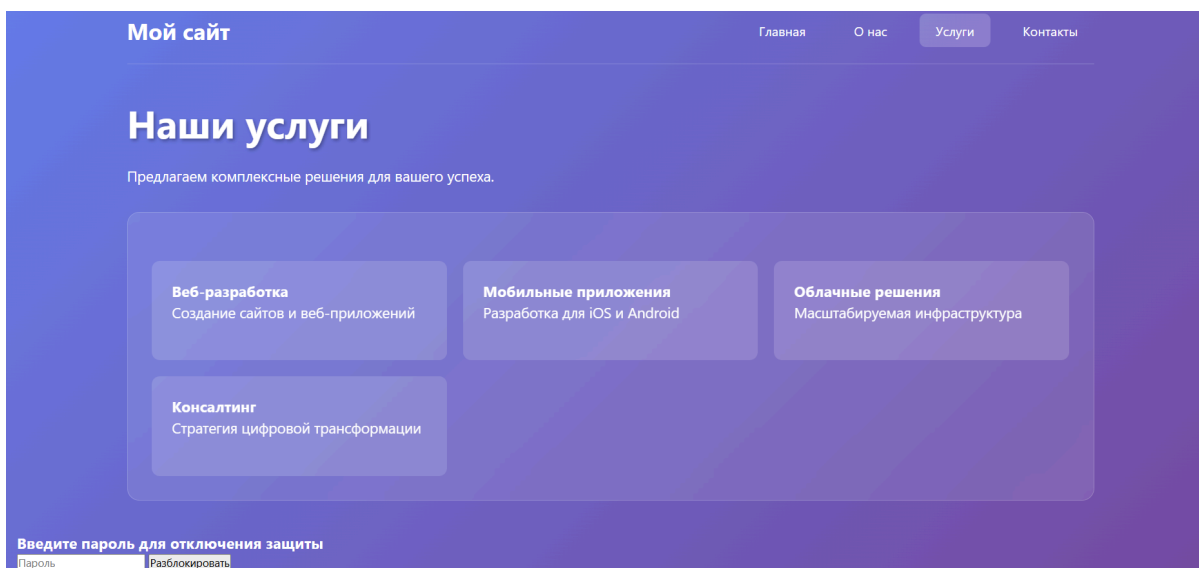


Рисунок 20 – Обзор раздела «Услуги» в режиме защиты

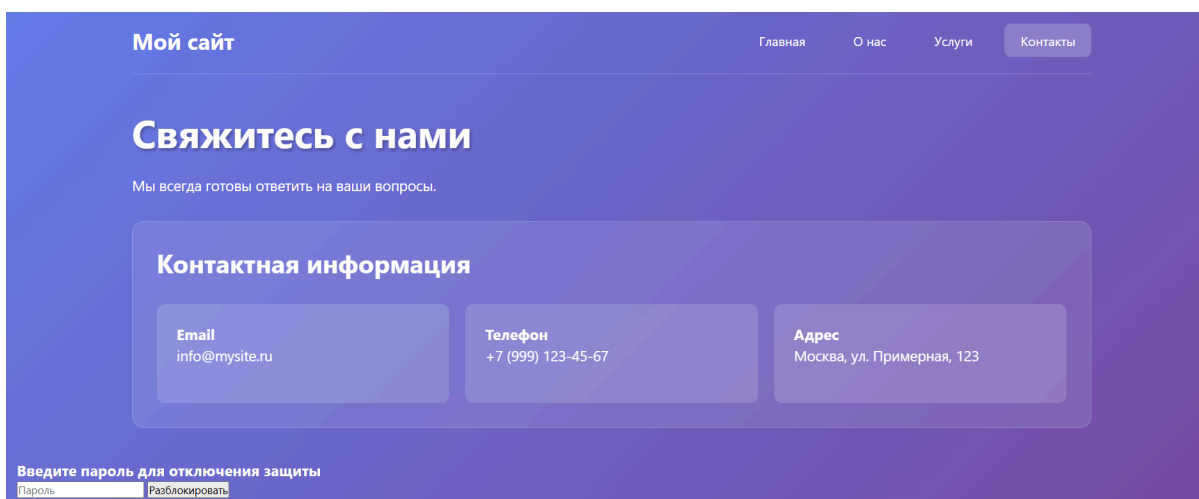


Рисунок 21 – Обзор раздела «Контакты» в режиме защиты

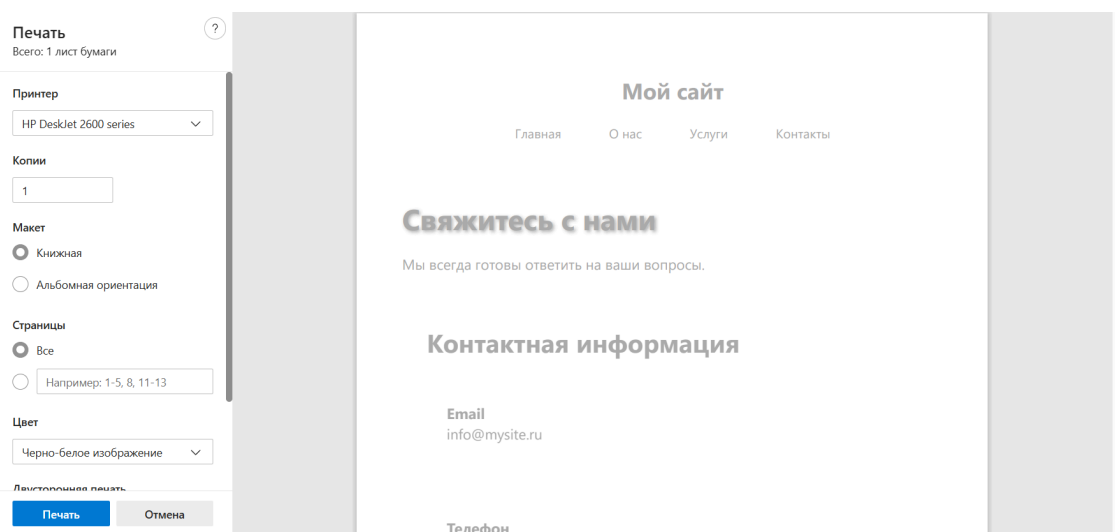


Рисунок 22 – Демонстрация работоспособности печати страниц

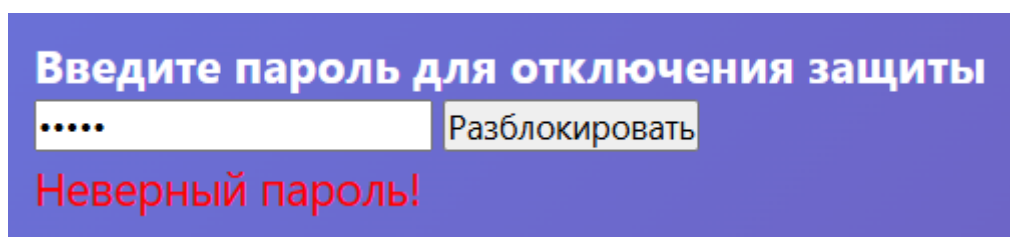


Рисунок 23 – Попытка снятия защиты с помощью неверного пароля

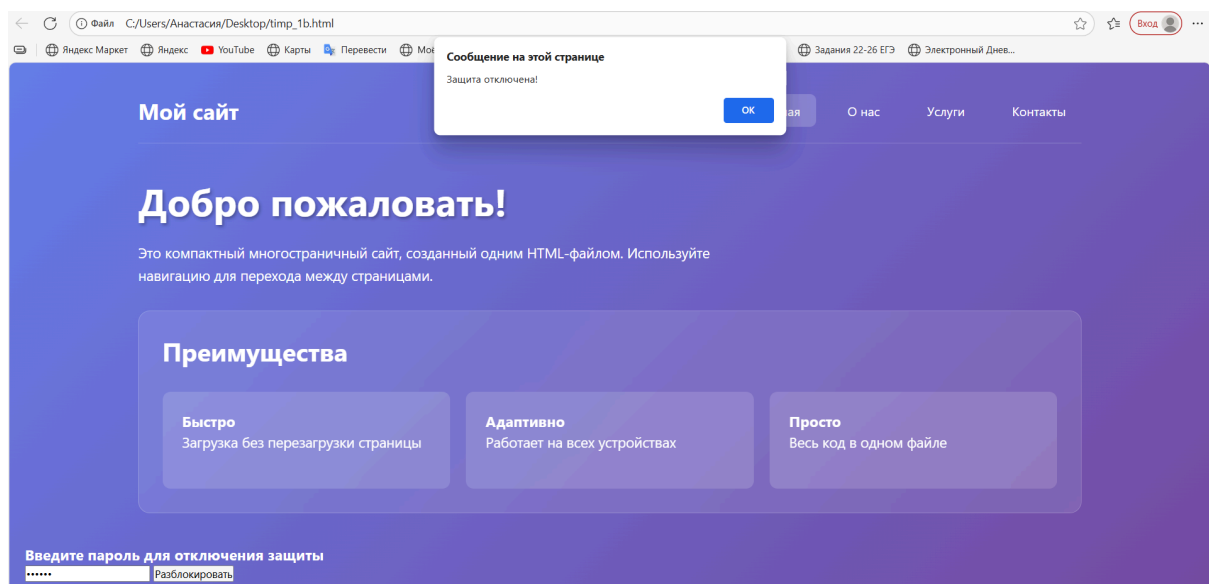


Рисунок 24 – Снятие защиты

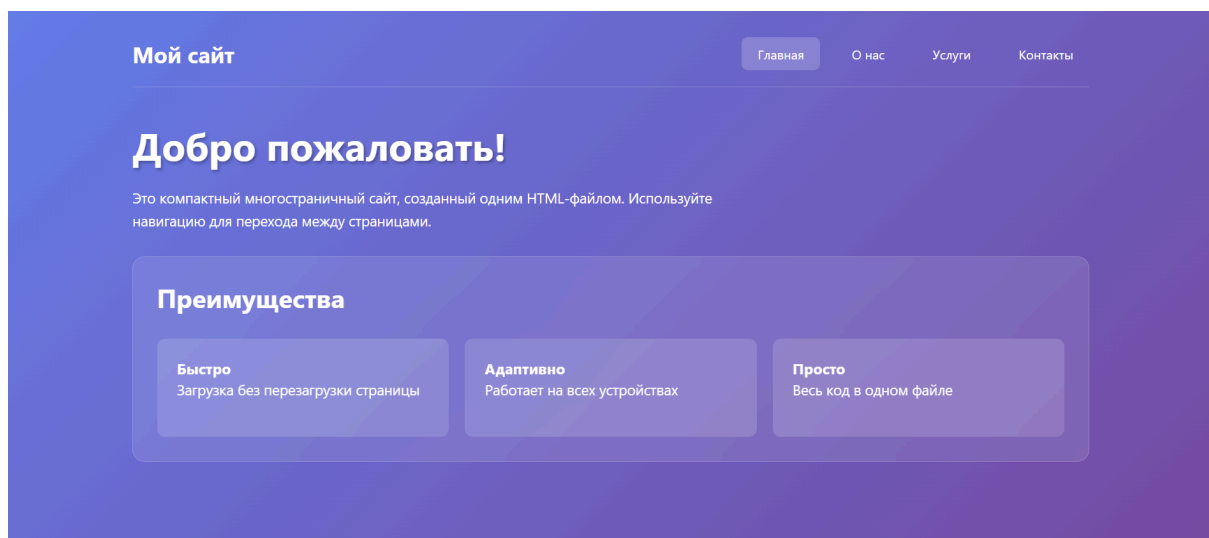


Рисунок 25 – Обзор раздела «Главная» после снятия защиты

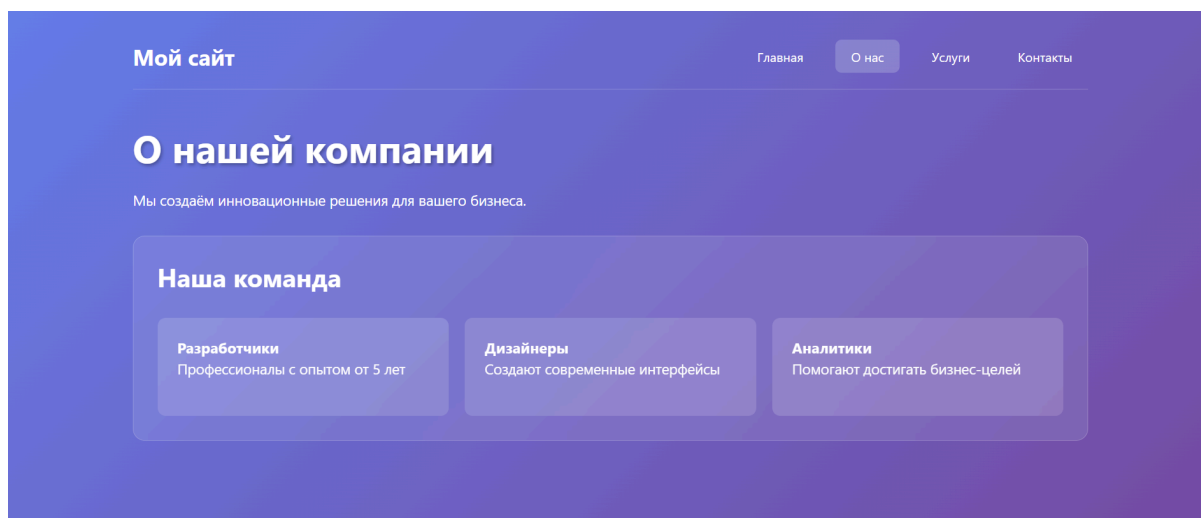


Рисунок 26 – Обзор раздела «О нас» после снятия защиты

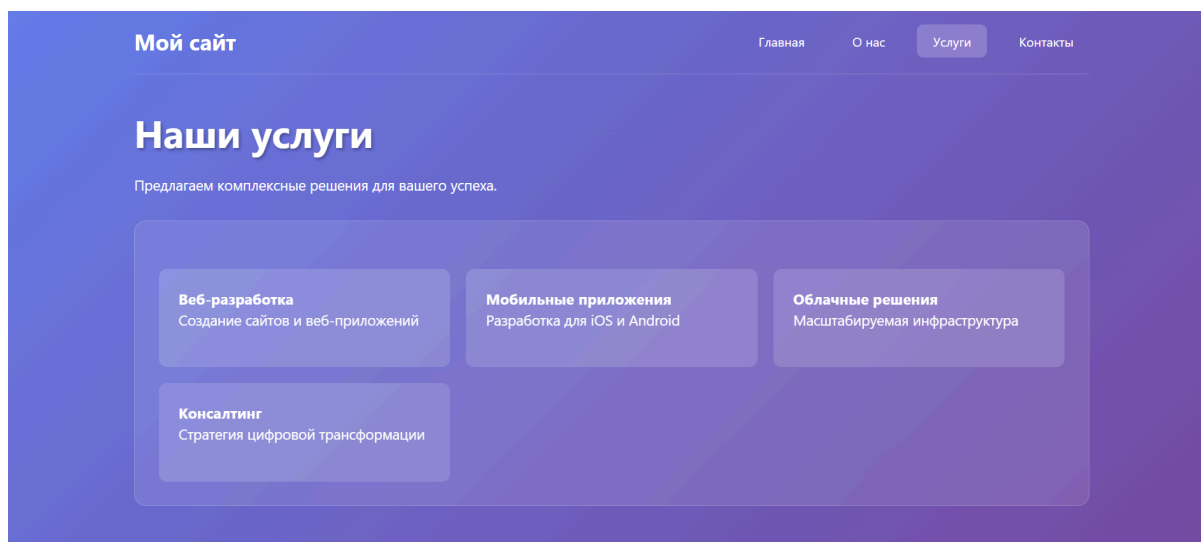


Рисунок 27 – Обзор раздела «Услуги» после снятия защиты

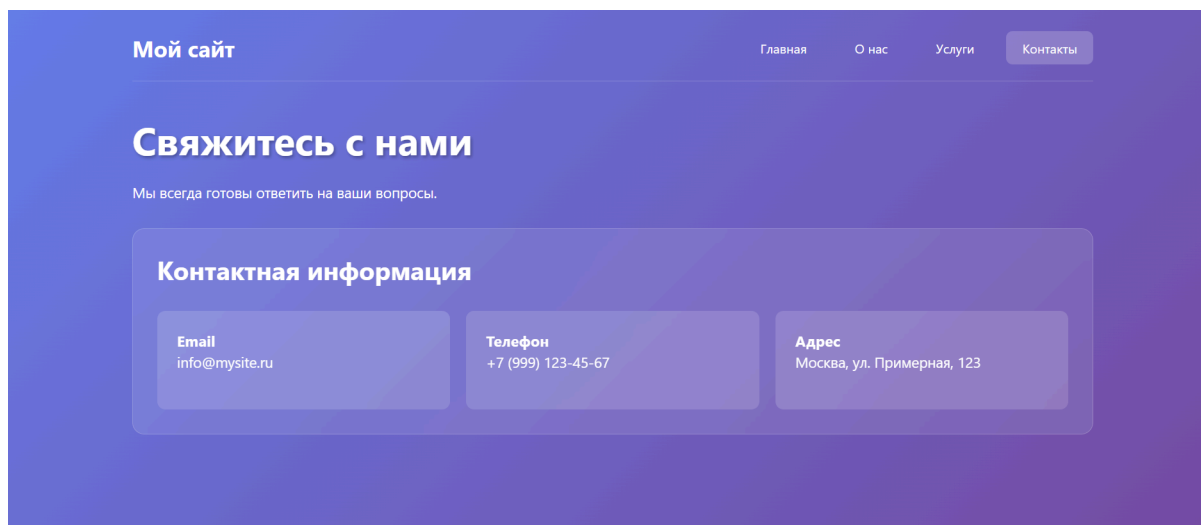


Рисунок 28 – Обзор раздела «Контакты» после снятия защиты

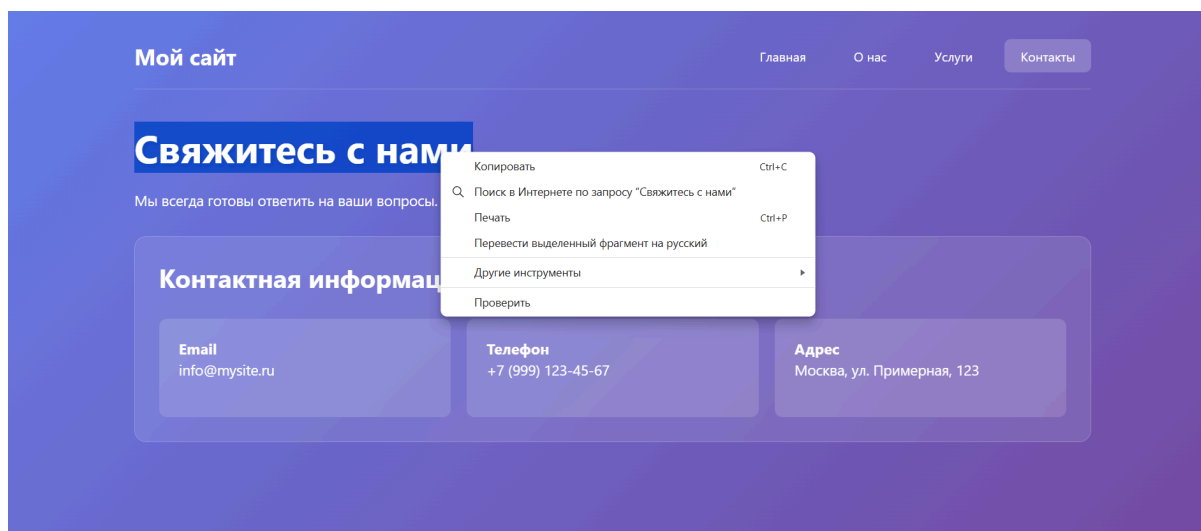


Рисунок 29 – Возможность выделения и копирования текста после снятия защиты

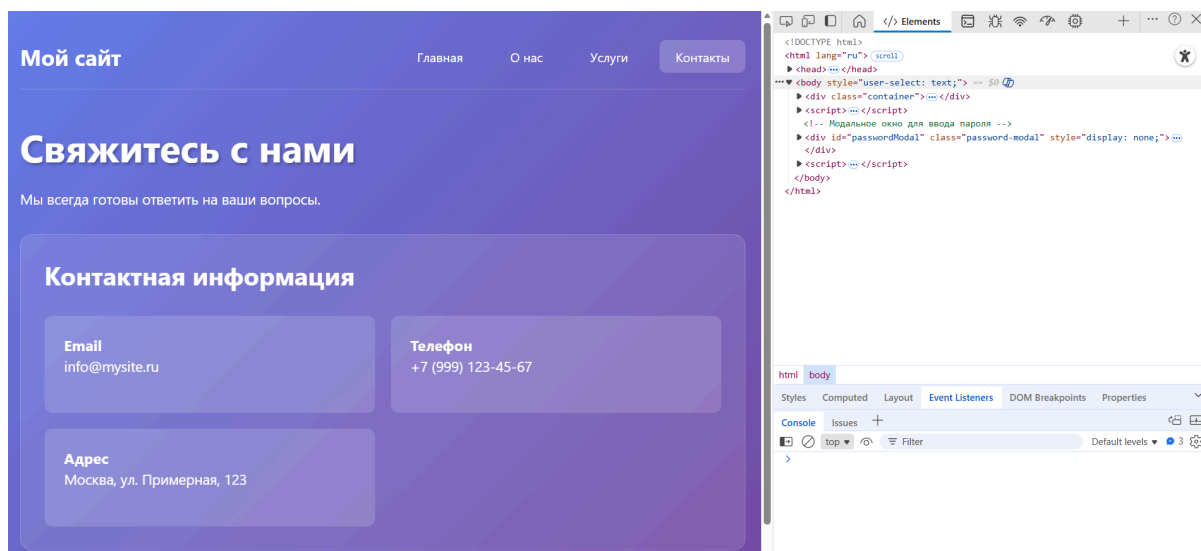


Рисунок 30 – Работа клавиши F12 после снятия защиты

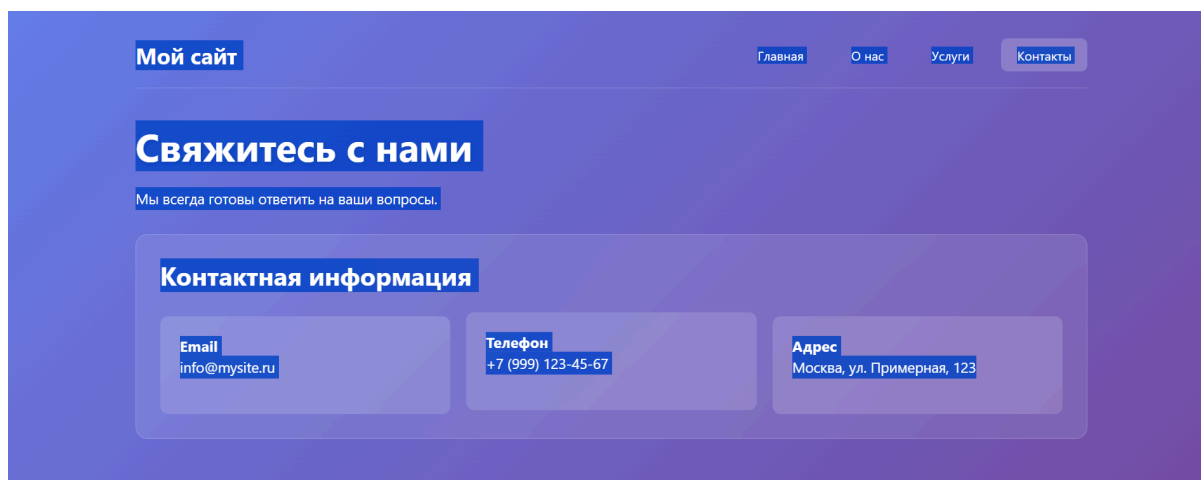


Рисунок 31 – Работа сочетания клавиш Ctrl+A после снятия защиты

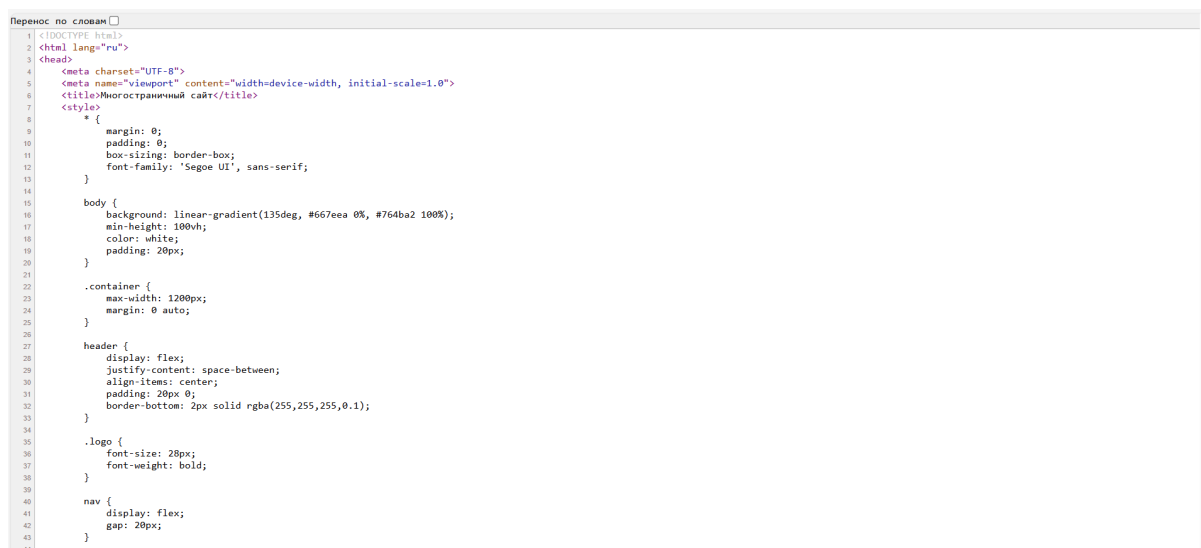


Рисунок 32 – Работа сочетания клавиш Ctrl+U после снятия защиты

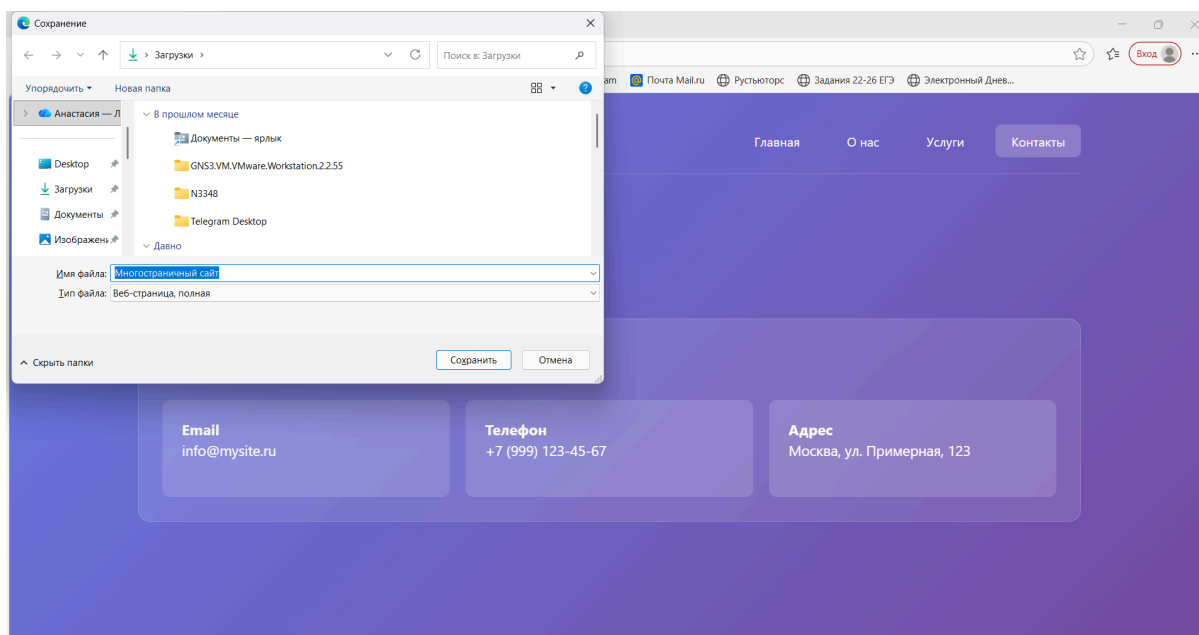


Рисунок 33 – Работа сочетания клавиш Ctrl+S после снятия защиты

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были успешно исследованы и реализованы практические методы защиты информации на двух различных уровнях: уровне файловой системы операционной системы и уровне веб-интерфейса. Цель работы – ограничение несанкционированного доступа, копирования и модификации данных – была достигнута посредством разработки двух независимых модулей защиты, демонстрирующих различные подходы к обеспечению информационной безопасности.

Оба модуля продемонстрировали эффективность применения различных технологических подходов к защите данных: первый – на уровне операционной системы с использованием механизмов контроля доступа и мониторинга файловой системы, второй – на уровне прикладного веб-интерфейса с использованием клиентских скриптов для ограничения пользовательских действий.

Проведенная работа позволила освоить на практике принципы контроля доступа к данным на разных уровнях, изучить соответствующие технологии защиты и реализовать функциональные решения, которые могут быть применены в реальных сценариях защиты информации от несанкционированного доступа и копирования.

Результаты работы подтверждают, что комбинирование различных уровней защиты – от файловой системы до пользовательского интерфейса – позволяет создавать более надежные и комплексные системы информационной безопасности, адаптированные под конкретные требования и условия эксплуатации.